

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к расчетной работе

по дисциплине «Проектирование программного обеспечения
интеллектуальных систем»:

**Найти тензорное произведение двух
неориентированных графов**

Студент:	В. В. Перминова
Группа:	321701
Руководитель:	М. Е. Садовский

Минск 2024

ВВЕДЕНИЕ

В современном мире графовые структуры играют ключевую роль в моделировании различных объектов и систем, от социальных сетей и маршрутизации в интернете до биологических и транспортных сетей. Одной из значимых задач, связанных с графами, является тензорное произведение графов. Этот процесс включает создание нового графа на основе двух исходных графов, где вершины и ребра формируются по определённым правилам.

В новом графе, полученном в результате тензорного произведения, каждая вершина представляется парой вершин из исходных графов. Ребро между двумя вершинами нового графа существует тогда и только тогда, когда в исходных графах существуют соответствующие ребра между соответствующими вершинами. Это позволяет создавать более сложные и многомерные структуры, которые могут глубже анализировать взаимосвязи между элементами сетей.

Тензорное произведение графов открывает возможности для исследования многомерных свойств сетей, оптимизации процессов, улучшения связности и повышения эффективности различных систем. Этот метод позволяет более детально изучать структуру графов и применять полученные результаты для решения практических задач в различных областях.

Цель работы — разработка агента для нахождения тензорного произведения двух заданных неориентированных графов. Агент должен автоматически определять вершины графов и связи, имеющиеся между ними, и формировать тензорное произведение в виде нового графа, содержащего все вершины и необходимые рёбра.

Для достижения поставленной цели необходимо решить следующие задачи:

- Разработать алгоритм, который будет эффективно извлекать все вершины из исходных графов.
- Реализовать механизм для проверки наличия ребер между вершинами, чтобы агент мог находить соответствующие связи в исходных графах.
- Обеспечить создание новых вершин и необходимых рёбер и добавление их в результирующий граф - тензорное произведение.
- Организовать формирование структурированного результата для последующего использования в других задачах.

Актуальность данной работы обусловлена тем, что графы широко используются в различных областях: анализ сетей, комбинаторика, теория графов, квантовые вычисления, машинное обучение и анализ данных, оптимизация и планирование. Автоматического поиска тензорного произведения графов может быть полезен для автоматизации анализа и обработки дан-

ных. Он позволяет быстро и эффективно находить тензорные произведения графов и использовать их для решения различных задач.

1 СПИСОК ИСПОЛЬЗОВАННЫХ ПОНЯТИЙ

а) **Неориентированный граф** (абсолютное понятие) - граф, в котором все ребра являются звеньями, то есть порядок двух концов ребра графа не существует

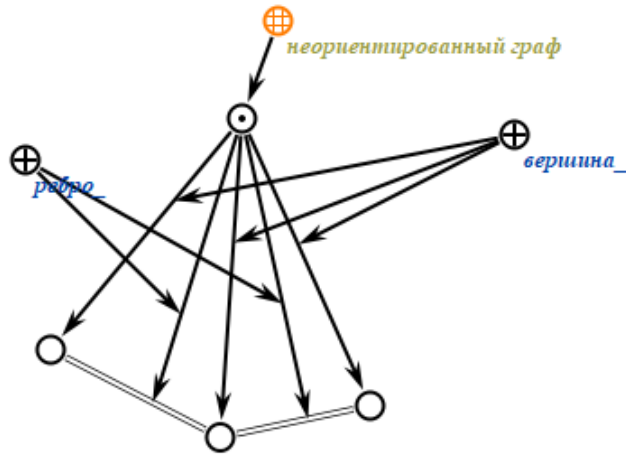


Рисунок 1.1 – Абсолютное понятие неориентированного графа

б) **Тензорное произведение двух неориентированных графов G и H** (абсолютное понятие) - граф, каждая вершина которого обозначает пару вершин оригинальных графов – одну от G и одну от H – при этом две вершины тензорного произведения соединены, если соединены как соответствующие им вершины в G , так и соответствующие им вершины в H .

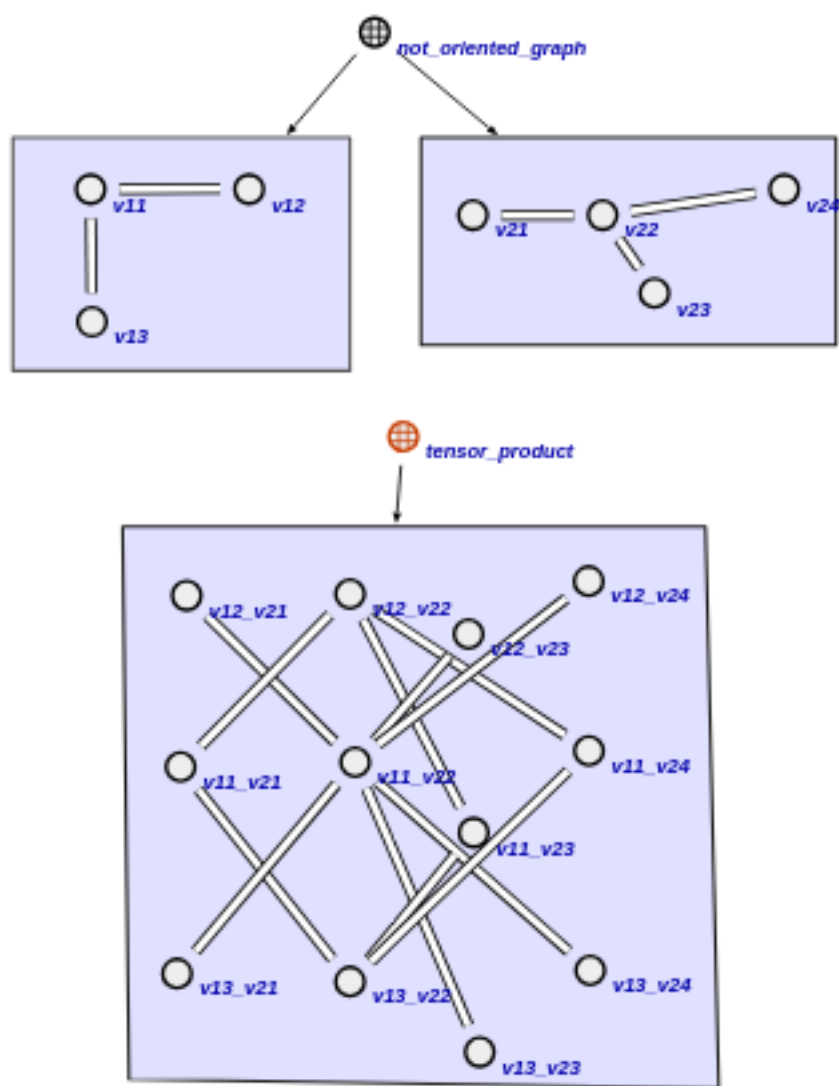


Рисунок 1.2 – Пример нахождения тензорного произведения

2 АНАЛИТИКА

Для решения задачи построения тензорного произведения двух неориентированных графов можно использовать различные подходы. В данной работе был разработан агент, реализующий алгоритм для поиска соответствующих связей в графах при помощи итераторов. В основе работы алгоритма лежит последовательный перебор всех рёбер двух графов.

Идея алгоритма заключается в следующем: агент перебирает все рёбра первого графа и для каждого из них перебирает все рёбра второго графа. Для каждой пары рёбер вызывается вспомогательная функция, которая создаёт вершины и рёбра графа тензорного произведения. Для этого в функцию передаются системные идентификаторы вершин, инцидентных с найденными рёбрами. Соответствующие исходные идентификаторы объединяются для создания новых. Если вершины с таким системным идентификатором ещё нет в графе тензорного произведения, добавляем её туда. В заключение работы функция создаёт ребро между созданными/найденными вершинами.

Недостатком метода является возможное увеличение времени обработки для очень больших графов, поскольку алгоритм обрабатывает все пары рёбер. Тем не менее, этот подход позволяет получить точный результат, который включает все возможные и необходимые вершины и связи между ними.

3 РАЗРАБОТКА

3.1 Глобальные переменные

Для работы алгоритма напрямую глобальные переменные не используются. Вся необходимая информация передается через параметры функций или инициализируется локально в пределах функций. Основные элементы, используемые в алгоритме, это:

- **m_context** - объект, представляющий контекст, с помощью которого агент взаимодействует с памятью и выполняет операции с элементами графа;
- **structure** - структура, которая используется для хранения тензорного произведения.

3.2 Алгоритм

Функция DoProgram - является основной функцией агента, которая реализует алгоритм создания тензорного произведения двух неориентированных графов. Она принимает в качестве аргумента *ScAction* & *action*, в котором содержится информация о графах, для которых необходимо построить тензорное произведение.

Шаги функции DoProgram:

- а) Получает аргументы *firstGraph* и *secondGraph* из действия *action*, представляющие собой адреса графов, для которых нужно построить тензорное произведение.
- б) Проверяет, существуют ли указанные элементы в памяти. Если хотя бы один из графов не существует, агент завершает выполнение с ошибкой.
- в) Создает пустую структуру *structure* для хранения тензорного произведения.
- г) С помощью итератора *ScIterator3Ptr* перебирает все рёбра первого графа.
 - д) Для каждого найденного ребра выполняет следующие действия:
 - 1) Получает *sourceAddr1* и *targetAddr1*, представляющие собой адреса вершин, которые инцидентны найдённом ребру.
 - 2) С помощью итератора *ScIterator3Ptr* перебирает все рёбра второго графа.
 - 3) Для каждого найденного ребра выполняет следующие действия:
 1. Получает *sourceAddr2* и *targetAddr2*, представляющие собой адреса вершин, которые инцидентны найдённом ребру.
 2. Вызывает вспомогательную функцию *AddEdge*.
- е) Передаёт *structure* в результат работы агента.

Функция AddEdge - вспомогательная функция, которая создаёт вершины и рёбра графа тензорного произведения. Она принимает на вход адреса четырёх вершин, которые инцидентны найденной паре рёбер, и структуры *structure*.

Шаги функции AddEdge:

- а) Проверяет правильность названия вершин.
- б) Создает *systemIdtf1*, который является суммой системных идентификаторов двух первых переданных вершин.
- в) Создает переменную *nodeAddr1*.
- г) Ищет вершину с идентификатором *systemIdtf1*
 - 1) Если такая вершина есть, присваиваем её адрес переменной *nodeAddr1*
 - 2) Иначе:
 - 1. Создаём пустую вершину *nodeAddr1*.
 - 2. Задаём созданной вершине системный идентификатор *systemIdtf1*.
 - 3. Помещаем эту вершину в *structure*.
- д) Создает *systemIdtf2*, который является суммой системных идентификаторов двух последних переданных вершин.
- е) Создает переменную *nodeAddr2*.
- ж) Ищет вершину с идентификатором *systemIdtf2*
 - 1) Если такая вершина есть, присваиваем её адрес переменной *nodeAddr2*
 - 2) Иначе:
 - 1. Создаём пустую вершину *nodeAddr2*.
 - 2. Задаём созданной вершине системный идентификатор *systemIdtf2*.
 - 3. Помещаем эту вершину в *structure*.
- з) Создает ребро *edgeAddr*, соединяющее вершины *nodeAddr1* и *nodeAddr2*.
- и) Помещает *edgeAddr* в *structure*.
- к) Возвращает полученный граф *structure*.

Алгоритм создания дополнения графа имеет сложность, где E — количество рёбер графа.

$$O(E^2) \tag{3.1}$$

4 ПОКАЗ ТЕСТОВЫХ СЦЕНАРИЕВ

а) Тест 1:

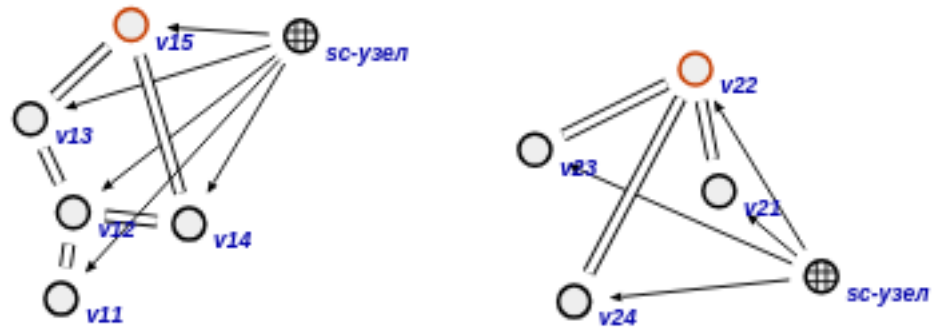


Рисунок 4.1 – Исходные графы 1 и 2

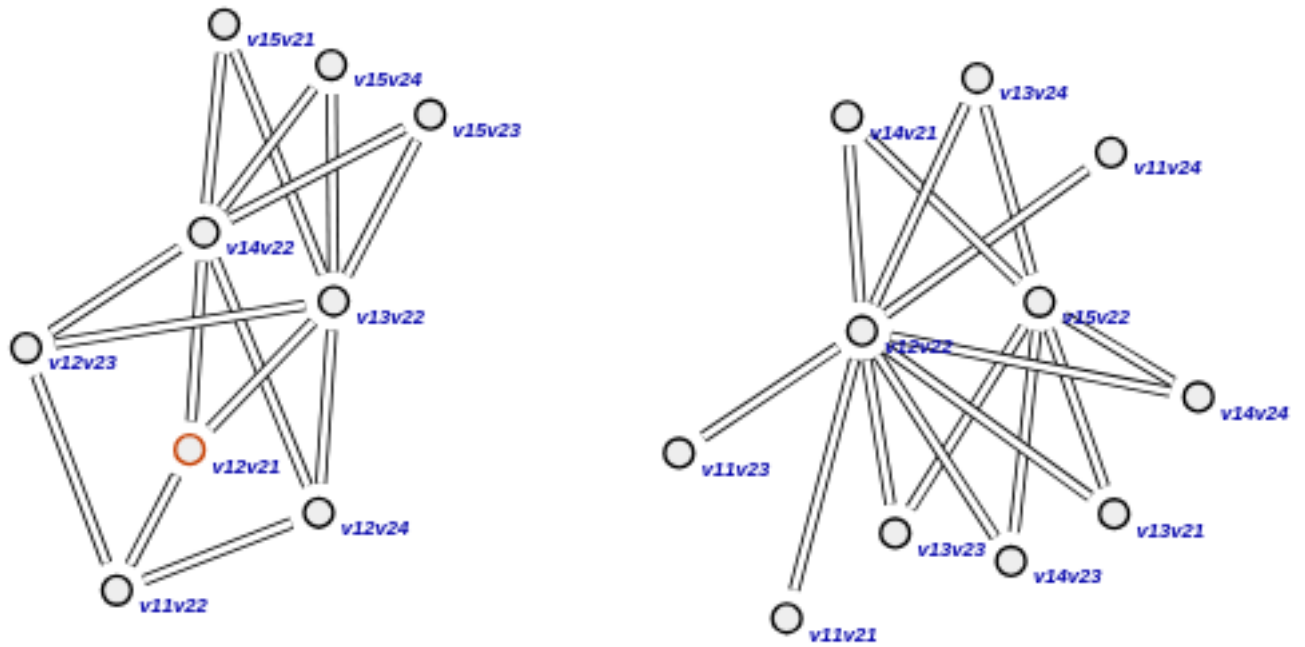


Рисунок 4.2 – Тензорное произведение графов 1 и 2

б) Тест 2:

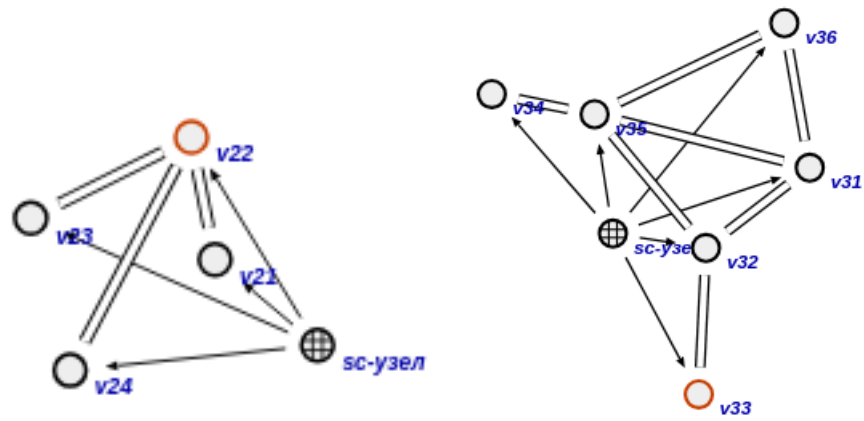


Рисунок 4.3 – Исходные графы 2 и 3

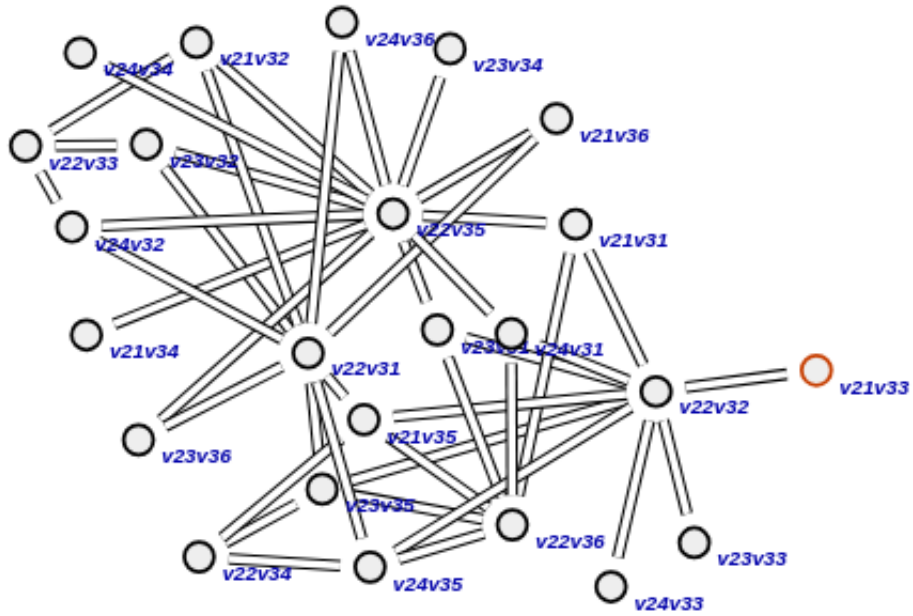


Рисунок 4.4 – Тензорное произведение графов 2 и 3

В) Тест 3:

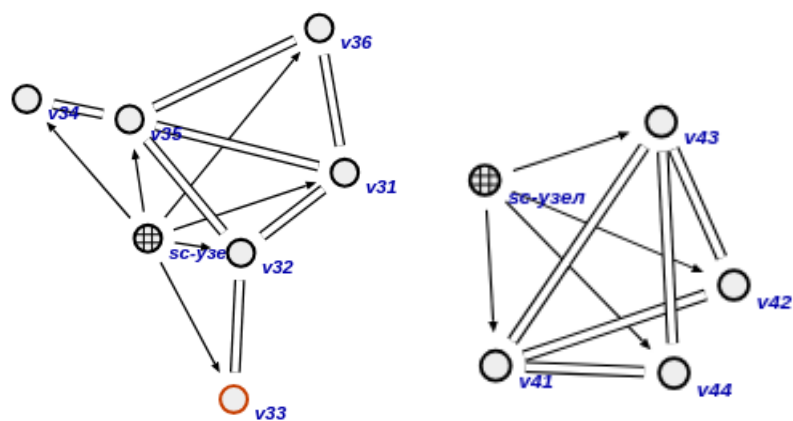


Рисунок 4.5 – Исходные графы 3 и 4

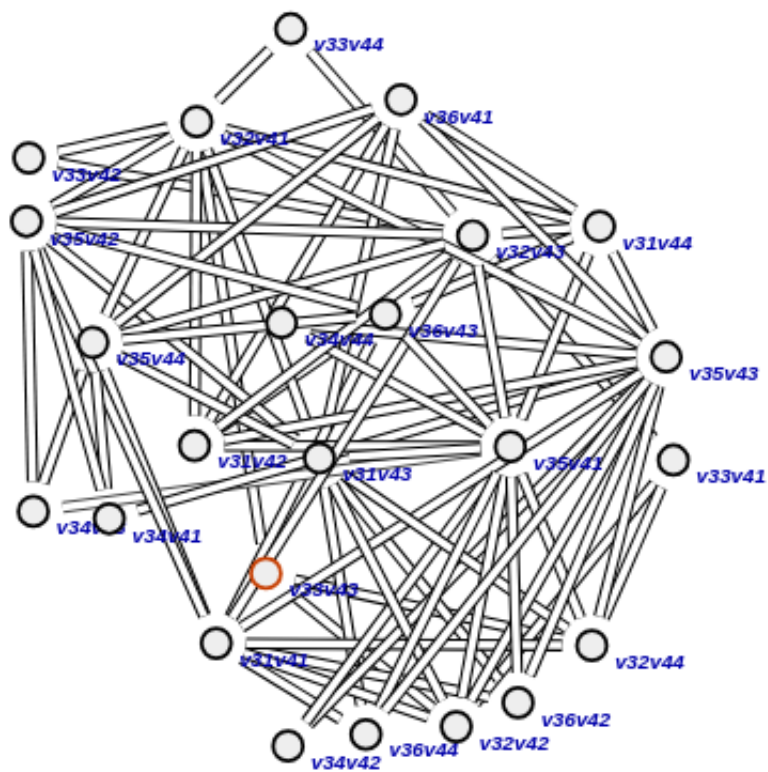


Рисунок 4.6 – Тензорное произведение графов 3 и 4

Г) Тест 4:

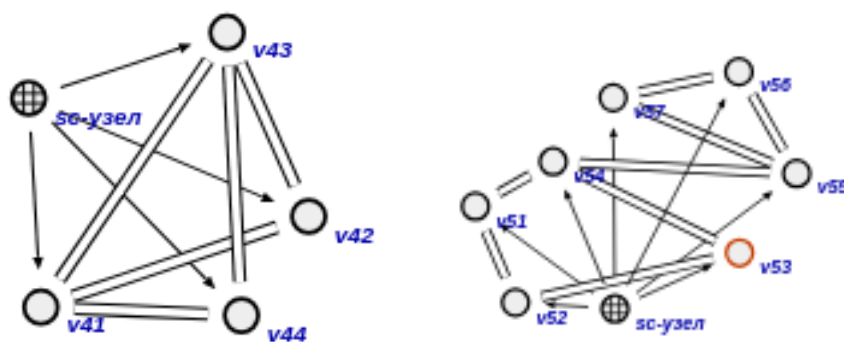


Рисунок 4.7 – Исходные графы 4 и 5

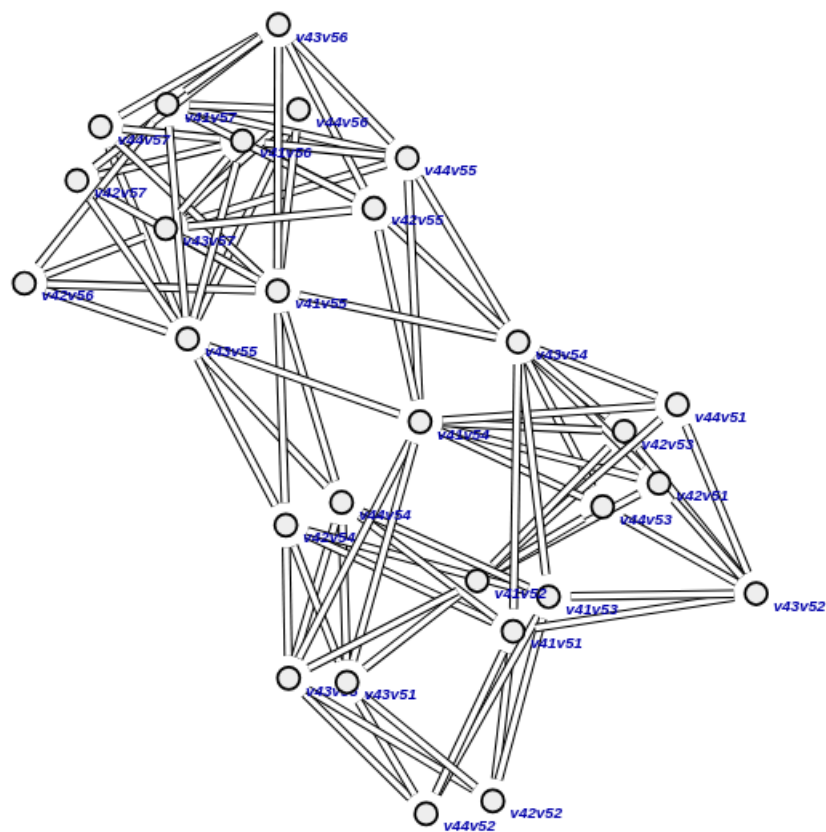


Рисунок 4.8 – Тензорное произведение графов 4 и 5

д) Тест 5:

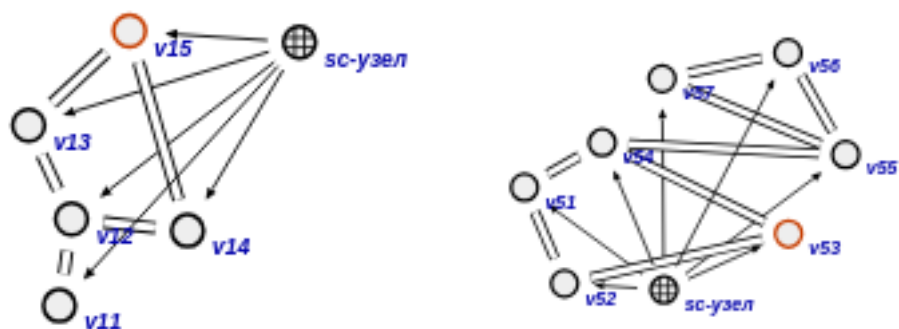


Рисунок 4.9 – Исходные графы 1 и 5

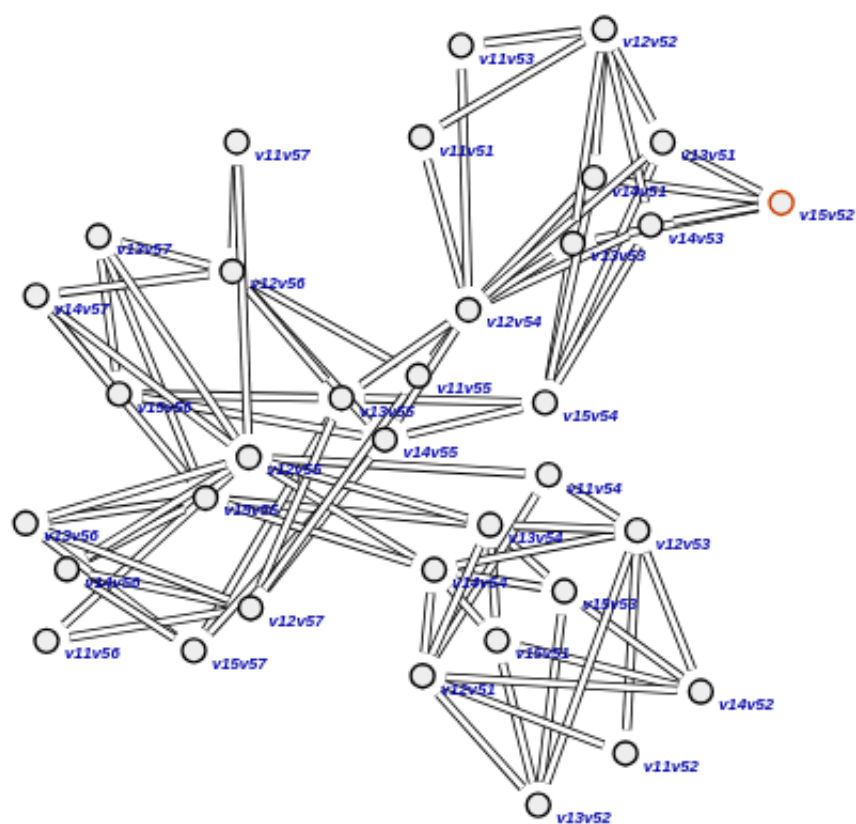


Рисунок 4.10 – Тензорное произведение графов 1 и 5

ЗАКЛЮЧЕНИЕ

В ходе данной работы был разработан агент для автоматического построения тензорного произведения двух заданных неориентированных графов. Основной целью проекта было создание инструмента, который бы позволил эффективно и точно выполнять операцию тензорного произведения, что является важным аспектом в теории графов и имеет широкое применение в различных областях, таких как анализ сетей, комбинаторика и квантовые вычисления.

Разработанная программа успешно справляется с поставленной задачей, обеспечивая корректное построение нового графа на основе двух исходных. В процессе работы были решены ключевые технические задачи, такие как обработка вершин и ребер графов.

Программа была протестирована на различных наборах данных, что подтвердило ее надежность и точность. Результаты тестирования показали, что разработанный инструмент может быть эффективно использован для анализа сложных сетей и решения задач, связанных с тензорным произведением графов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Харари, Фрэнк. Теория графов / Фрэнк Харари. — 1973.
- [2] Иванова, ГС. Полная характеристика структуры неориентированного графа / ГС Иванова, ВА Овчинников. No. 4. — Некоммерческое партнерство «Национальный Электронно-Информационный Консорциум», 2016. — Рр. 106–123.
- [3] sc-machine [Электронный ресурс]. - Режим доступа: <https://ostis-ai.github.io/sc-machine>. - Дата доступа: 01.12.2024.